

Advanced Machine Learning

DATA 642

Lecture 8 — A popular Tensor Decomposition Algorithm

*Figures and notes are from the paper "Introduction to Tensor Decompositions and their Applications in Machine Learning," (<https://arxiv.org/pdf/1711.10781>)

Tucker Decomposition

The Tucker decomposition decomposes a tensor into a so-called core tensor and multiple matrices which correspond to different core scalings along each mode. Therefore, the Tucker decomposition can be seen as a higher-order PCA (Principal Component Analysis).

Intuitive Explanation

- **Core Tensor:** Think of the core tensor as a compressed version of the original tensor. It captures the essence of the data but in a smaller and more manageable form.
- **Factor Matrices:** The matrices (often called factor matrices) associated with each mode of the tensor represent the principal components in that mode. These matrices transform the core tensor back to the original tensor dimensions when multiplied together.

Why Higher-Order PCA?

- **PCA Analogy:** In PCA, we decompose a matrix into a set of orthogonal components that capture the variance in the data. Similarly, Tucker decomposition breaks down a higher-order tensor (which is a multi-dimensional array) into a core tensor and a set of matrices that capture the variance along each dimension (or mode) of the tensor.
- **Dimensionality Reduction:** Just like PCA reduces the dimensions of a matrix by identifying the most significant components, Tucker decomposition reduces the dimensions of a tensor by identifying the most significant interactions between its elements. This makes it easier to analyze and visualize complex data.

Practical Benefits

- **Compression:** By choosing the dimensions of the core tensor to be smaller than the original tensor, Tucker decomposition effectively compresses the data. This is useful for storage and computational efficiency.
- **Interpretability:** The factor matrices provide a way to understand the underlying structure of the data. Each column in a factor matrix can be seen as a principal component that explains a certain amount of variance in the data along that mode.
- **Flexibility:** Unlike some other tensor decomposition methods, Tucker decomposition is flexible in choosing the size of the core tensor, allowing for different levels of compression and detail depending on the needs of the analysis.

Visualization

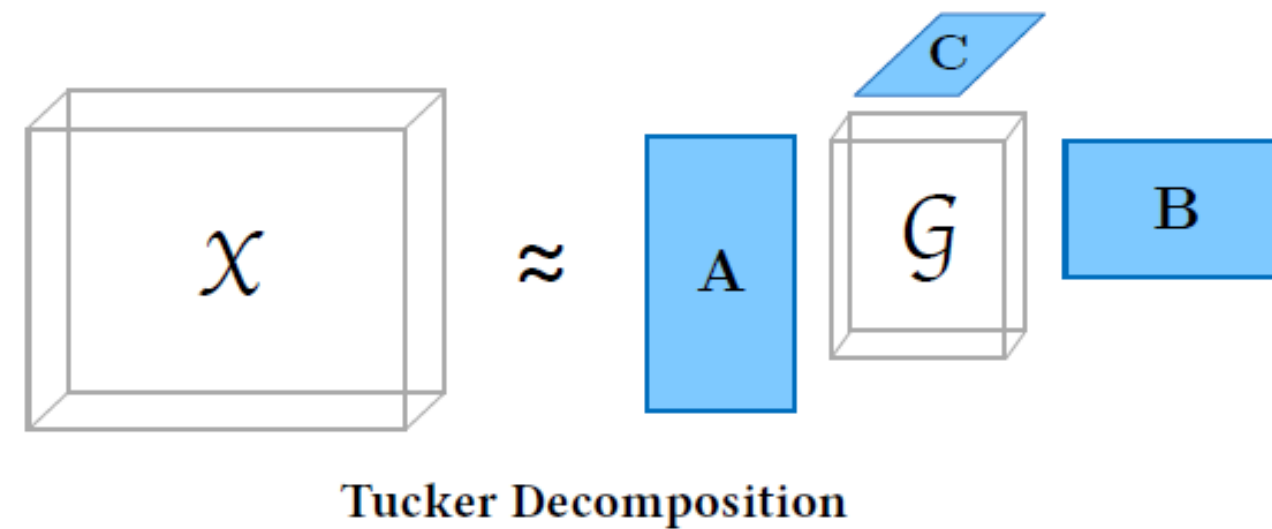
- **Core Tensor:** Imagine the core tensor as a smaller, dense block that captures the main interactions within the data.
- **Factor Matrices:** Each factor matrix can be visualized as a set of axes that stretch or compress the core tensor to reconstruct the original data.

Problem formulation

In the 3-way tensor case, we can express the problem of finding the Tucker decomposition of a tensor $\mathcal{X} \in \mathbb{R}^{I \times J \times K}$ with $\mathcal{G} \in \mathbb{R}^{P \times Q \times R}$, $\mathbf{A} \in \mathbb{R}^{I \times P}$, $\mathbf{B} \in \mathbb{R}^{J \times Q}$, $\mathbf{C} \in \mathbb{R}^{K \times R}$ as follows:

$$\min_{\hat{\mathcal{X}}} \|\mathcal{X} - \hat{\mathcal{X}}\| \text{ with } \hat{\mathcal{X}} = \sum_{p=1}^P \sum_{q=1}^Q \sum_{r=1}^R g_{pqr} \mathbf{a}_p \odot \mathbf{b}_q \odot \mathbf{c}_r = \mathcal{G} \times_1 \mathbf{A} \times_2 \mathbf{B} \times_3 \mathbf{C} \quad (1)$$

A graphical view is given by the following figure



In the previous setting:

- $\mathcal{G}$ is the core tensor, which expresses how and to what extent different tensor elements interact with each other.
- The factor matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ are often referred to as the principal components in the respective tensor mode.
- We can already see that if we pick $P < I$, $Q < J$, and $R < K$, this will result in a compression of $\mathcal{X}$, with $\mathcal{G}$ being the compressed version of $\mathcal{X}$.

The matricized version

$$\hat{X}_{(1)} = \mathbf{A}G_{(1)}(\mathbf{C} \otimes \mathbf{B})^T \quad (2)$$

$$\hat{X}_{(2)} = \mathbf{B}G_{(1)}(\mathbf{C} \otimes \mathbf{A})^T \quad (3)$$

$$\hat{X}_{(3)} = \mathbf{C}G_{(1)}(\mathbf{B} \otimes \mathbf{A})^T \quad (4)$$

where $\otimes$ denotes the Kronecker product.

The general case

In the general N-way case we get:

$$\hat{\mathcal{X}} = \sum_{r_1=1}^{R_1} \sum_{r_2=1}^{R_2} \cdots \sum_{r_N=1}^{R_N} g_{r_1 r_2 \cdots r_N} \mathbf{a}_{i_1 r_1}^{(1)} \odot \cdots \odot \mathbf{a}_{i_N r_N}^{(N)} = \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \cdots \times_N \mathbf{A}^{(N)} \quad (5)$$

$$\hat{X}_{(n)} = \mathbf{A}^{(n)} G_{(n)} (\mathbf{A}^{(N)} \otimes \cdots \otimes \mathbf{A}^{(n+1)} \otimes \mathbf{A}^{(n-1)} \otimes \cdots \otimes \mathbf{A}^{(1)})^T \quad (6)$$

Before we can look into the exact computation of the Tucker decomposition, we first need to introduce the concept of n-rank.

- The n-rank of a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$ corresponds to the column rank of the n-th unfolding of the tensor $X_{(n)}$.
- Formally, this is denoted $\text{rank}_n(\mathcal{X})$.

It is important to not confuse this concept with the tensor rank.

Higher Order Singular Value Decomposition (HOSVD)

Concept:

- HOSVD leverages the fact that for a given tensor $\mathcal{X}$, we can find an exact Tucker decomposition of rank $(R_1, R_2, \dots, R_N)$ where $R_n = \text{rank}_n(\mathcal{X})$.
- This results in the Higher Order Singular Value Decomposition (HOSVD).

Key Idea:

- The main idea behind HOSVD is to find the components that best capture the variation in each mode (n) independently, without considering the other modes at that moment.
- This is analogous to the basic PCA concept, applied to each mode of the tensor separately.

Algorithm: HOSVD:

- $\text{HOSVD}(\mathcal{X}; R_1, \dots, R_N)$

1. **for** $n = 1, \dots, N$ **do**

 ▪ $\mathbf{A}^{(n)} \leftarrow R_n$ leading left singular vectors of $X_{(n)}$

end for

2. Compute the core tensor $\mathcal{G}$ as:

$$\mathcal{G} = \mathcal{X} \times_1 \mathbf{A}^{(1)T} \times_2 \cdots \times_N \mathbf{A}^{(N)T} \quad (7)$$

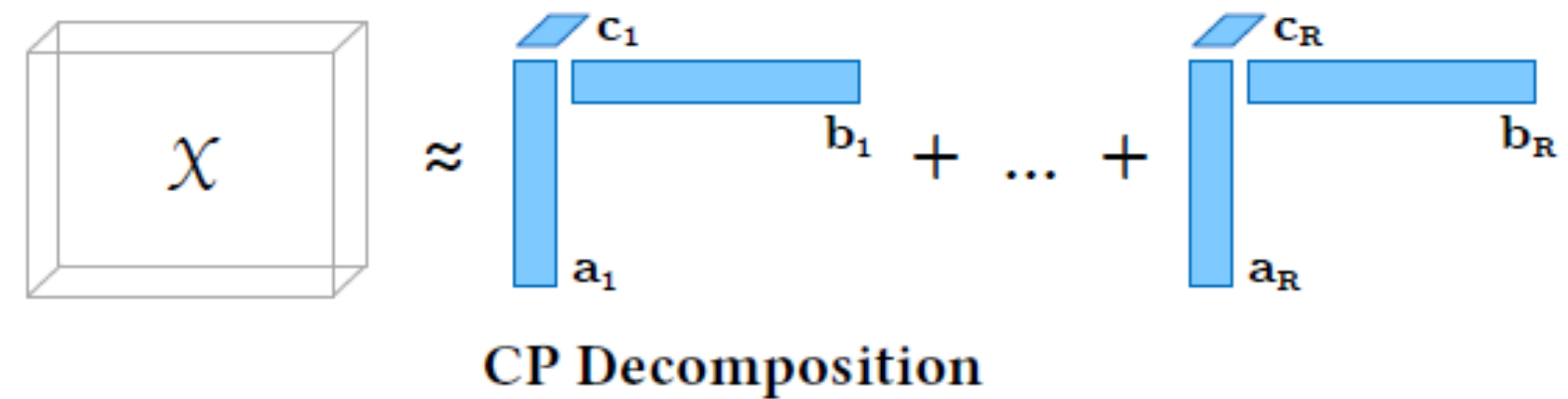
3. Return $\mathcal{G}, \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)}$

Limitations of Tucker decomposition

- **Computational Complexity:** Tucker decomposition involves computing the factor matrices and the core tensor, which can be computationally expensive for large tensors, especially when the ranks are high.
- **Memory Requirements:** The storage requirements for the core tensor and factor matrices can be substantial, particularly for tensors with high ranks or large dimensions.
- **Dependence on Initializations:** The quality of the Tucker decomposition can depend on the initialization of factor matrices, and convergence to the optimal solution may not always be guaranteed.

Canonical Polyadic Decomposition (CPD)

An alternative to Tucker decomposition is CPD which is a type of rank decomposition that expresses a tensor as the sum of a finite number of rank-one tensors. CPD has its roots in two main decompositions: **the Canonical Decomposition (CANDECOMP)** and the **Parallel Factors (PARAFAC) decomposition**.



Tucker Decomposition

- **Flexibility:** Tucker decomposition offers more flexibility in capturing complex interactions between tensor elements by allowing different ranks along each mode.
- **Interpretability:** The core tensor in Tucker decomposition provides insight into how different dimensions interact with each other, which can be valuable for understanding the underlying structure of the data.
- **Compression:** Tucker decomposition can compress the data more effectively than CPD, especially when the interactions between dimensions are complex and cannot be adequately captured by CPD.

Canonical Polyadic Decomposition (CPD)

- **Simplicity:** CPD is simpler and computationally more efficient compared to Tucker decomposition, making it preferable for large-scale datasets or applications where computational resources are limited.
- **Interpretability:** While Tucker decomposition provides insight into interactions between dimensions, CPD offers a more straightforward interpretation with factor matrices representing patterns within each mode.
- **Scalability:** CPD may scale better to higher-dimensional data or larger datasets due to its simpler structure and lower computational complexity compared to Tucker decomposition.